MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

SCHOOL OF ELECTRICAL ENGINEERING

NEUROSLIDE
FPGA CNN ACCELERATOR FOR POTHOLE DETECTION

Minor Specialization Report

Submitted by

Venkata Durgesh	**Gurudeep Sai**	**Manudeep**
230907454	230907202	230907436

Under the guidance of

Dr. Rahul Ratnakumar

in partial fulfilment of the requirements for the award of the degree of

MINOR SPECIALIZATION IN

Embedded Systems

MANIPAL INSTITUTE OF TECHNOLOGY
(A Constituent Unit of Manipal Academy of Higher Education)
MANIPAL-576104, KARNATAKA, INDIA
November 2026

ACKNOWLEDGMENTS

We express our gratitude to our project guide, Dr. Rahul Ratnakumar, for guidance during the development of NeuroSlide. The project brought together neural-network modelling, SystemVerilog design, FPGA implementation and embedded software integration, and the discussions during these stages helped us connect the algorithm with its hardware realization.

We thank the Director, Dean, Associate Deans, project coordinators and faculty of the School of Electrical Engineering, Manipal Institute of Technology, for the academic support and facilities available for this work. We also thank the laboratory incharge and technical staff for their assistance with the development environment and FPGA experimentation.

We acknowledge the collective effort of Venkata Durgesh, Gurudeep Sai and Manudeep in continuing the project through model development, hardware design, verification and demonstration. We are grateful to everyone who offered technical feedback during the project.

ABSTRACT

Road-surface image classification is a useful component of automated pothole monitoring. A practical embedded classifier must balance recognition accuracy with computation, memory use and response time. This project develops NeuroSlide, a compact convolutional neural network and a custom FPGA accelerator for classifying 64 × 64 grayscale images as normal road or pothole. The work focuses on translating a trained model into a streaming hardware architecture using local data reuse, fixed-point arithmetic and pipeline registers. A throughput of 240 frames per second is the design objective; it is treated as a target rather than an experimentally established result.

The final software model contains eight first-layer filters, two four-channel groups in the second convolution layer, 1,568 flattened features and a single output neuron, giving 1,945 trainable parameters. The recorded software evaluation reports 89.20% accuracy on 750 test images. SystemVerilog modules implement BRAM input access, line buffers, sliding windows, convolution, activation, pooling and dense accumulation. The design was integrated with the processing system of a Zynq-7000 ZedBoard through AXI GPIO, exported as an XSA platform and launched from Vitis. Implementation screenshots report 14,164 LUTs, 4,345 registers, 6.5 BRAM tiles and 220 DSP blocks, with positive timing slack under the shown constraints. The exported clock configuration and the 50 MHz CNN timing constraint require reconciliation before operating-frequency and frame-rate claims can be finalized. The completed prototype demonstrates the development path from model design to FPGA deployment and software-controlled result indication.

TABLE OF CONTENTS

Section	Page
Acknowledgments	i
Abstract	ii
List of tables	iv
List of figures	v–vi
List of abbreviations	vii
Chapter 1 Introduction	1
Chapter 2 Literature review and objectives	3
Chapter 3 Methodology	5
Chapter 4 Contribution of each student	23
Chapter 5 Result analysis	24
Chapter 6 Conclusion and future scope	32
Visible outcome	34
References	35
Minor specialization project details	37
Plagiarism report	38

LIST OF TABLES

No	Table title	Page
1.1	Evolution from proposals to the final model	2
2.1	Literature lessons applied to NeuroSlide	4
3.1	Development components and their roles	5
3.2	Final model shapes and trainable parameters	7
3.3	Algorithmic multiplication counts per image	8
3.4	Numerical formats on the final datapath	10
3.5	Active RTL hierarchy	15
3.6	Expected stage counts for the final model	16
3.7	Hardware handoff values used by software	20
3.8	Deployment events recorded by XSDB	22
5.1	Class-wise software performance	25
5.2	Implemented device utilization	26
5.3	Arithmetic distribution in the custom hierarchy	26
5.4	Reported timing values	27
5.5	Warning counts visible in the implementation record	28
5.6	Analytical performance quantities	30
5.7	Priority checks before final technical sign-off	31
6.1	Visible technical deliverables	34

LIST OF FIGURES

No	Figure title	Page
3.1	Development flow from model preparation to board indication	6
3.2	Four-channel groups in the second convolution layer	8
3.3	Recorded training and validation accuracy curves	9
3.4	Recorded training and validation loss curves	9
3.5	Line-buffer reuse and the sliding 3 by 3 window	11
3.6	Original first-stage parallel-processing design sketch	12
3.7	Original multichannel second-stage design sketch	13
3.8	Dense accumulation and final binary decision	14
3.9	Recorded functional simulation of BRAM and row-buffer activity	15
3.10	Recorded simulation of the stream entering second-layer buffers	16
3.11	Hierarchical utilization from the implemented design	17
3.12	Captured 20 ns clock constraint	18
3.13	Clock summary containing both 100 MHz and 50 MHz entries	18
3.14	Custom IP identity in the packaging interface	19

LIST OF FIGURES CONTINUED

No	Figure title	Page
3.15	Packaged accelerator output ports	19
3.16	Exported system structure with Zynq PS AXI GPIO and CNN IP	20
3.17	Vitis standalone application structure	21
5.1	Software test metrics and confusion matrix	24
5.2	Resource utilization calculated from the implementation screenshot	26
5.3	Reported implementation timing summary	27
5.4	Per-clock setup and hold results	27
5.5	Vivado implementation power estimate	28
5.6	ZedBoard connected for the hardware demonstration	29
5.7	Development computers and FPGA during deployment	29

LIST OF ABBREVIATIONS

Abbreviation	Expansion
ANN	Artificial Neural Network
AXI	Advanced eXtensible Interface
BRAM	Block Random Access Memory
CNN	Convolutional Neural Network
DDR	Double Data Rate memory
DRC	Design Rule Check
DSP	Digital Signal Processing block
ELF	Executable and Linkable Format
FPGA	Field-Programmable Gate Array
FPS	Frames Per Second
GPIO	General-Purpose Input/Output
HDL	Hardware Description Language
IP	Intellectual Property block
JTAG	Joint Test Action Group interface
LUT	Lookup Table
MAC	Multiply–Accumulate
MIO	Multiplexed Input/Output
PL	Programmable Logic
PS	Processing System
ReLU	Rectified Linear Unit
RTL	Register-Transfer Level
SoC	System on Chip
TNS	Total Negative Slack
WHS / WNS	Worst Hold Slack / Worst Negative Slack
XSA / XSDB	Xilinx Support Archive / Xilinx System Debugger

CHAPTER 1
INTRODUCTION

1.1 Background and problem statement

Potholes create irregular road surfaces that can be identified from images, but their appearance changes with lighting, shadows, road texture and viewing angle. The task considered here is binary image classification: for each prepared image, the system decides whether a pothole is present. It does not estimate depth, generate a bounding box or determine a geographical location. Those functions would require additional sensing or model outputs.

Convolutional neural networks learn spatial filters that respond to useful visual patterns. Early layers can represent local edges and texture, while later layers combine these responses into features that support a decision [1z. For an FPGA implementation, the model must also be expressed as finite-width arithmetic, memory transactions and clocked control. The engineering problem is therefore to preserve the model's data ordering and numerical meaning while mapping its operations into a resource-constrained device.

1.2 Motivation and relevance

NeuroSlide investigates this mapping through a deliberately small network. A fixed 64 × 64 input reduces the amount of storage and permits explicit line-buffer dimensions. Small 3 × 3 kernels support repeated use of nearby pixels, and pooling reduces the volume of data passed to later stages. A hardware pipeline can overlap these operations once enough valid data is available.

The project's motivation is a 240 FPS processing target, corresponding to one frame every 4.167 ms. This is a design requirement for future measurement, not evidence that all other vision processors operate at 60–120 FPS. A fair comparison would require the same model, image dimensions, input-transfer method and measurement boundary.

1.3 Main engineering contribution

The work joins model selection, trained-weight export, fixed-point arithmetic, streaming RTL, physical implementation, IP integration and a standalone C application. Its value lies in building and inspecting this complete path rather than treating an accurate software classifier as a finished embedded system. Later chapters distinguish software accuracy, implementation reports and physical deployment because each demonstrates a different property.

1.4 Scope and evolution of the design

The architecture changed as the hardware work progressed. The earlier assignment explored padded feature maps, three road categories and a possible systolic approach. A subsequent streaming report described valid convolutions and a larger second layer. The final model summary and active top module establish the smaller grouped binary network used as the reference architecture in this report.

Table 1.1 Evolution from proposals to the final model

Stage	Architecture described	Interpretation
Initial assignment	64 → 32 → 16 spatial path; 8 then 16 channels; three categories	Conceptual proposal with padding and a proposed systolic approach
Streaming report	62 → 31 → 29 → 15; 16 second-layer filters; 3,600 features	Intermediate design narrative with selected paths verified
Final model and active RTL	62 → 31 → 29 → 14; 8 grouped second-layer filters; 1,568 features	Binary classifier used for final model discussion

The final network uses valid convolution. Its second pooling stage takes complete 2 × 2 windows from a 29 × 29 map and produces 14 × 14 outputs. It does not use the earlier proposal's padding to obtain 15 × 15. Likewise, the final dense stage produces one signed score, and the hardware decision compares that score with zero.

1.5 Demonstration boundary

The demonstrator reads an image from internal BRAM and produces a classification bit in programmable logic. The processor reads this bit through AXI GPIO and drives a board LED. The current application reads the result once after a fixed delay. It does not implement a live camera interface, processor-to-accelerator image transfer, repeated-frame scheduling or an interrupt-driven completion protocol.

The physical deployment record identifies a ZedBoard. Portability through IP packaging is part of the design approach, but a controlled multi-board comparison is outside the verified results. The 89.20% figure is a software test result and is not presented as measured fixed-point FPGA classification accuracy.

CHAPTER 2

LITERATURE REVIEW AND OBJECTIVES

2.1 Neural networks and road-image classification

Goodfellow, Bengio and Courville describe convolution, parameter sharing and pooling as central components of image-oriented neural networks [1]. These ideas motivate using the same small filter across many image positions. In this project, that reuse also has a hardware interpretation: one stored kernel operates repeatedly on windows assembled from buffered rows.

Maeda et al. demonstrate road-damage detection using images captured through smartphones [2]. Their work establishes road imagery as a useful learning-based application, but it addresses a broader detection task. NeuroSlide instead uses binary classification of prepared low-resolution images. The published detector's accuracy or frame rate cannot be transferred to this prototype.

2.2 Efficient neural-network hardware

The survey by Sze et al. explains that efficient inference depends on data movement, memory hierarchy, precision and parallel execution as well as arithmetic [3]. This motivates local line-buffer reuse and the separate examination of LUTs, DSP blocks and storage. A small parameter count alone is insufficient to predict whether a design will fit or meet timing.

Eyeriss introduces a row-stationary dataflow that reduces movement of weights, activations and partial sums [4]. NeuroSlide is not an implementation of Eyeriss. The relevant lesson is the importance of retaining reusable information near arithmetic units. Its line buffers keep previous image rows available without requiring nine independent external reads for every 3 × 3 window.

FINN demonstrates FPGA inference using a streaming architecture for binarized neural networks [5]. NeuroSlide uses wider fixed-point values and manually written RTL rather than the FINN flow. The comparison supports the choice to connect layer-specific datapaths, while also showing that throughput depends on the slowest stage and the amount of parallel hardware assigned to it.

2.3 Numerical precision

Jacob et al. describe integer-arithmetic inference and training methods that preserve accuracy after quantization [6]. The present design uses different binary-point positions in different stages. This makes bit-accurate checking essential: a correct floating-point classifier and a correctly routed FPGA design are not sufficient to prove numerical equivalence.

2.4 Platform and implementation literature

The Zynq-7000 technical reference manual describes the processing system, programmable-logic interfaces, clocks and GPIO resources [7]. AXI GPIO provides an AXI4-Lite register interface to general-purpose signals [8]. These resources support a simple prototype in which a one-bit CNN output is readable by processor software.

AMD's constraints guide explains how timing requirements are expressed for synthesis and implementation [9]. A clock constraint describes timing analysis; it does not create a frequency divider. The IP-packaging guide describes how custom logic can be packaged and reused [10], while the Vitis embedded-development documentation covers the software platform and application workflow [11]. These documents provide the implementation background for the integration stages.

Table 2.1 Literature lessons applied to NeuroSlide

Source	Design lesson	Project response
[1], [2]	Learn spatial features from road images	Small binary CNN on 64 × 64 grayscale inputs
[3], [4]	Reuse data near compute units	BRAM input, line buffers and window registers
[5], [6]	Match precision and parallelism to hardware	Streaming modules with stage-specific fixed-point formats
[7]–[11]	Integrate and constrain the complete system	Zynq PS–PL connection, AXI GPIO and Vitis application

2.5 Project objectives

1. Develop and evaluate a compact CNN for binary pothole classification.

2. Export trained parameters with explicit numerical scale and feature ordering.

3. Implement line buffers, sliding windows, convolution, activation, pooling and dense accumulation in SystemVerilog.

4. Verify stage behavior and optimize the design for the selected FPGA resources and a 50 MHz CNN timing target.

5. Package the accelerator, integrate it with Zynq processing-system resources and demonstrate software-readable output on hardware.

6. Establish the measurements needed to assess the 240 FPS target, numerical equivalence and repeated-frame operation.

CHAPTER 3

METHODOLOGY

3.1 Type and nature of the project

This is a combined software-development, digital-hardware-design and hardware-experimentation project. Software work covers image preparation, network training and evaluation, parameter export and the standalone C application. Hardware work covers RTL design, functional simulation, synthesis, placement, routing, timing analysis and integration with the Zynq processing system.

The algorithm is described at the tensor level before it is mapped to clocked modules. The mapping defines the value format, channel order, spatial counters and valid signals for every interface. The completed system is then assessed through separate software, RTL, implementation and deployment evidence rather than one undifferentiated success criterion.

3.2 Hardware and software requirements

Table 3.1 Development components and their roles

Component	Specification or observed identity	Purpose
Target board	ZedBoard; xc7z020clg484-1 in XSA	Zynq processing system and FPGA fabric
Model environment	Google Colab; notebook screenshot Cnn_weights_8_8.ipynb	Model construction and software evaluation
Hardware language	Verilog and SystemVerilog	Clocked arithmetic and control
Implementation tool	Vivado 2025.2 in XSA metadata	Synthesis, implementation and hardware export
Embedded tool	Vitis and XSDB 2026.1 in launch log	Standalone application and JTAG launch
On-chip memory	Block Memory Generator instance	Read-only input-image stream
Interface IP	AXI GPIO, SmartConnect, Processor System Reset	PS access and reset distribution
Output indication	PS GPIO MIO7 as identified in C code	Steady or blinking LED

The exported platform contains the bitstream, hardware handoff and processing-system initialization files. The original notebook, complete Vivado project and BRAM initialization file are not part of the reproducible source package described here; archiving them is an important completion step.

3.3 Development workflow

The workflow begins with the image representation and a small trainable model. Once the output shapes are fixed, trained weights are exported into hardware include files. The RTL is organized into modules that correspond to the major transformations, with additional storage and control to enforce the stream order.

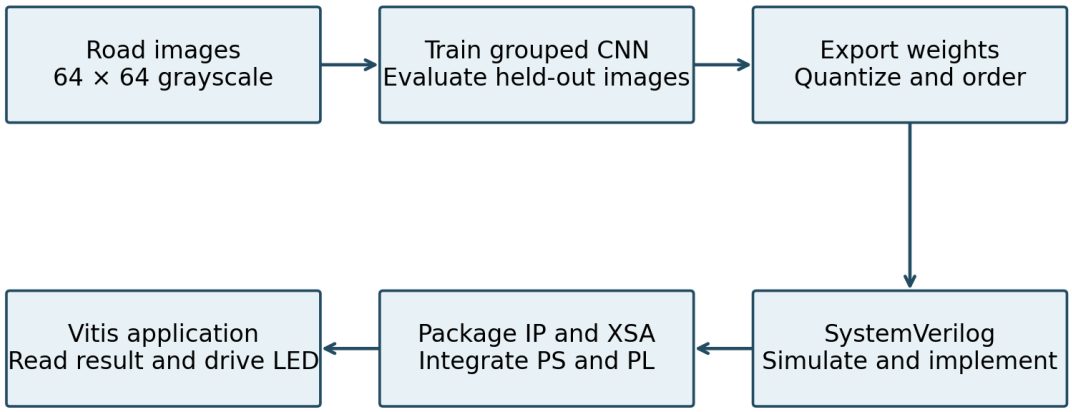


Figure 3.1 Development flow from model preparation to board indication

Functional simulation is used to observe addresses, row buffers, window movement and intermediate results before physical implementation. Synthesis maps the arithmetic and storage into FPGA resources. Placement and routing then determine whether the mapped circuit can satisfy the declared timing requirements. The successful implementation is exported to the software environment for deployment.

3.4 Image preparation and model construction

The input representation is a single-channel 64 × 64 image. The earlier assignment describes RGB-to-grayscale conversion and resizing. In the current input-memory module, pixels are stored as 16-bit values with 16 fractional bits, representing normalized intensities. Host-side preprocessing and memory initialization must use the same normalization as the trained model.

The final model is named FPGA_Pothole_CNN in the recorded model summary. The training record contains accuracy and loss curves and a binary classification report. Dataset provenance, exact training and validation counts, optimizer settings and augmentation details must be retained with the notebook before the experiment can be independently reproduced; they are not inferred from the plotted curves.

3.5 Final network architecture

Table 3.2 Final model shapes and trainable parameters

Operation	Output shape	Parameters
Input	64 × 64 × 1	0
Conv 1 with 8 filters	62 × 62 × 8	80
Pool 1 with stride 2	31 × 31 × 8	0
Channel split	Two groups of 31 × 31 × 4	0
Conv 2 group 1	29 × 29 × 4	148
Conv 2 group 2	29 × 29 × 4	148
Concatenate and ReLU	29 × 29 × 8	0
Pool 2 with stride 2	14 × 14 × 8	0
Flatten	1,568	0
Dense binary output	1	1,569
Total	One classification score	1,945

A valid 3 × 3 convolution with stride one reduces each spatial dimension by two. Thus, the first convolution produces 62 × 62 values per filter, followed by 31 × 31 after pooling. The second convolution produces 29 × 29. Its complete 2 × 2 pooling windows produce floor(29/2) = 14 output positions along each dimension.

Each first-layer filter has nine weights and one bias, giving 8 × (9 + 1) = 80 parameters. Each second-layer group has four output filters, each connected to four input channels: 4 × (3 × 3 × 4 + 1) = 148 parameters per group. The dense layer adds 1,568 weights and one bias. These counts agree with the recorded model summary.

3.6 Grouped convolution and computational cost

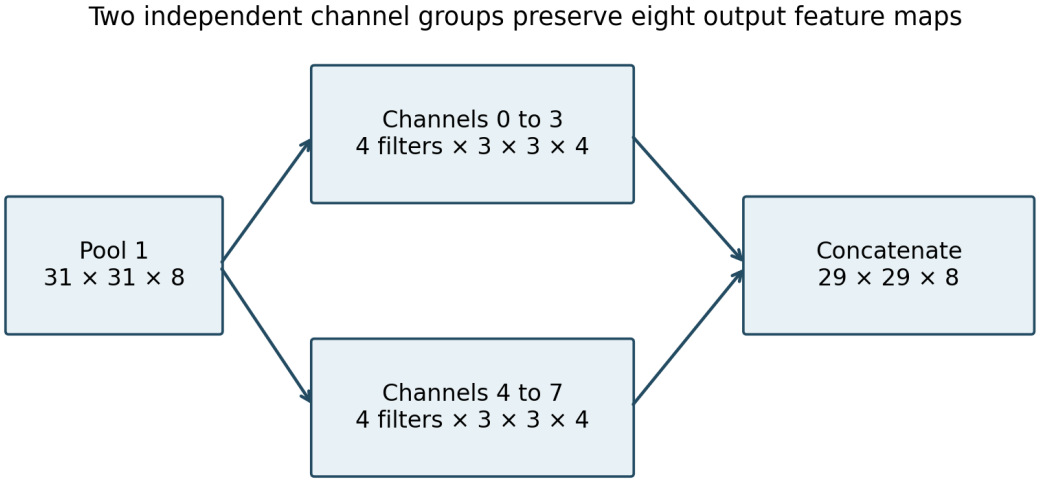


Figure 3.2 Four-channel groups in the second convolution layer

In the final second layer, output channels 0–3 use input channels 0–3, while output channels 4–7 use input channels 4–7. This is grouped convolution. It is not depthwise convolution, because each output filter combines four input channels. It also differs from a dense eight-input-channel convolution in which every output sees all eight channels.

Table 3.3 Algorithmic multiplication counts per image

Stage	Calculation	Multiplications
Conv 1	62 × 62 × 8 × 9	276,768
Grouped Conv 2	29 × 29 × 8 × 4 × 9	242,208
Dense	14 × 14 × 8	1,568
Total	Excludes biases, activation and comparisons	520,544

For the same eight second-layer outputs, full channel connectivity would require 484,416 convolution multiplications. The grouped layer halves this count to 242,208. This is an analytical comparison of model operations, not a measured twofold speedup. The implemented cycle count also depends on scheduling, pipeline gaps, memory behavior and the amount of arithmetic mapped in parallel.

The grouping must be preserved during training and weight export. Taking a trained dense convolution and simply dropping half its channel connections would define a different model. The final software summary explicitly contains two grouped branches and a concatenation operation, matching the intended hardware grouping.

3.7 Training and testing procedure

Model training and validation are shown in the supplied notebook captures. The curves span 50 epochs. Training accuracy rises rapidly and then increases more slowly, while validation accuracy improves in several steps. Training loss continues to decrease; validation loss remains higher and changes less smoothly. This makes the held-out test result more informative than the training curve alone.

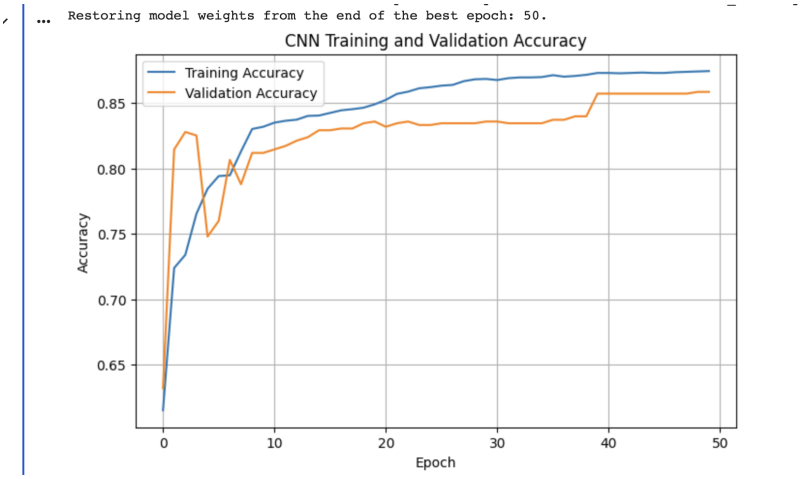


Figure 3.3 Recorded training and validation accuracy curves

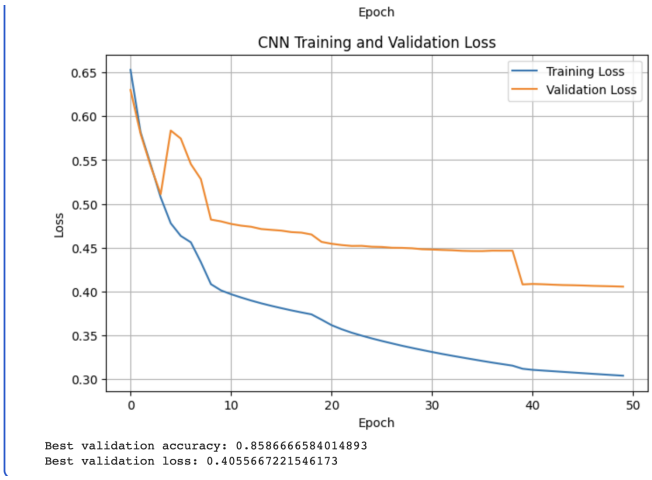


Figure 3.4 Recorded training and validation loss curves

The separate test report contains 375 normal-road images and 375 pothole images. Its confusion matrix is analyzed in Chapter 5. These figures describe the software model. A fixed-point hardware implementation needs an additional comparison on the same images to quantify numerical and classification differences.

3.8 Parameter export and fixed-point representation

The final top module selects layer1_8_weights.vh, layer2_8_weights.vh and ann_8.vh through its child modules. The files contain 72 first-layer kernel coefficients plus eight biases, 288 grouped second-layer coefficients plus eight biases, and 1,568 dense weights plus one bias. The total is 1,945 stored learned values.

Table 3.4 Numerical formats on the final datapath

Value	Stored width	Fractional bits	Interpretation
Input pixel	16 unsigned	16	Normalized nonnegative intensity
Window and Conv 1 weights	18 signed	16	Window is widened; weights use Q2.16
Pool 1 interface	24 signed	8	Requantized feature values
Conv 2 weights and biases	10 signed	8	Q2.8 learned parameters
Pool 2 interface	18 signed	8	Feature values for ANN
Dense weights and bias	18 signed	16	Q2.16 learned parameters
Dense accumulator	46 signed	24	Products and aligned bias

Here, the width and fractional-bit count define the format unambiguously; Q notation follows the comments in the source, with the sign included in the nonfractional part. A stored integer q with F fractional bits represents q/2^F. Multiplication adds the two fractional-bit counts, so scale alignment is required before adding a bias or narrowing a result.

A weight of 0.25 becomes 16,384 at a scale of 2^16 and 64 at a scale of 2^8. Arithmetic right shifts are used to reduce fractional precision in the datapath. Truncation and overflow can affect the result, so the exported integer values and intermediate widths must be checked against a bit-accurate reference, not only against floating-point outputs.

3.9 Input memory and line-buffer design

The line_buffer module instantiates blk_mem_gen_0 as a read-only memory. A 12-bit address counter covers the 4,096 image locations. The pixel signal is 16 bits wide, while the three row buffers and the 3 × 3 output window use signed 18-bit elements. Widening a nonnegative 16-bit pixel provides room for signed multiplication without interpreting its most significant intensity bit as a negative sign.

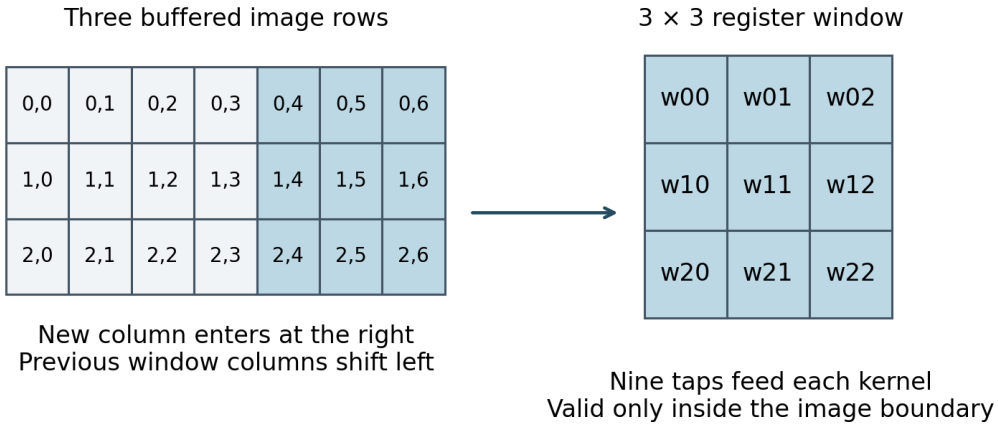


Figure 3.5 Line-buffer reuse and the sliding 3 by 3 window

Each row buffer contains 64 locations. For a given column index, the incoming pixel enters the first buffer, the previous first-buffer value moves to the second, and the previous second-buffer value moves to the third. Three taps then feed the right-hand side of the window, while existing window columns shift left. Because these are clocked nonblocking assignments, downstream logic observes registered values from the preceding cycle.

The memory contains 4,096 × 16 = 65,536 input bits, or 8 KiB. The three 64-element row buffers contain 3 × 64 × 18 = 3,456 logical bits. These are storage calculations, not a prediction of exact BRAM inference. Physical memory usage depends on primitive widths, ports, inference and other buffers in the design.

Row and column counters suppress invalid edge windows. The source includes an initial delay to accommodate data arrival. First-window timing and the final memory reads need cycle-by-cycle verification because a valid flag must identify the same pixels that the arithmetic uses.

3.10 First convolution and pooling layer

The layer1_8 module applies eight sets of nine kernel coefficients to the shared input window. Sharing the window avoids eight independent copies of the image-memory reader. The arithmetic is divided into registered partial sums, followed by a combined result and scale adjustment. This breaks a long arithmetic path into clocked stages.

After the convolution result is formed, nonnegative activation values enter pooling buffers. For each channel, two rows of 62 positions support a 2 × 2 pooling window. Max pooling selects the largest of four values and emits an output at the required stride. The resulting first-layer interface carries eight 24-bit feature values and layer1_valid.

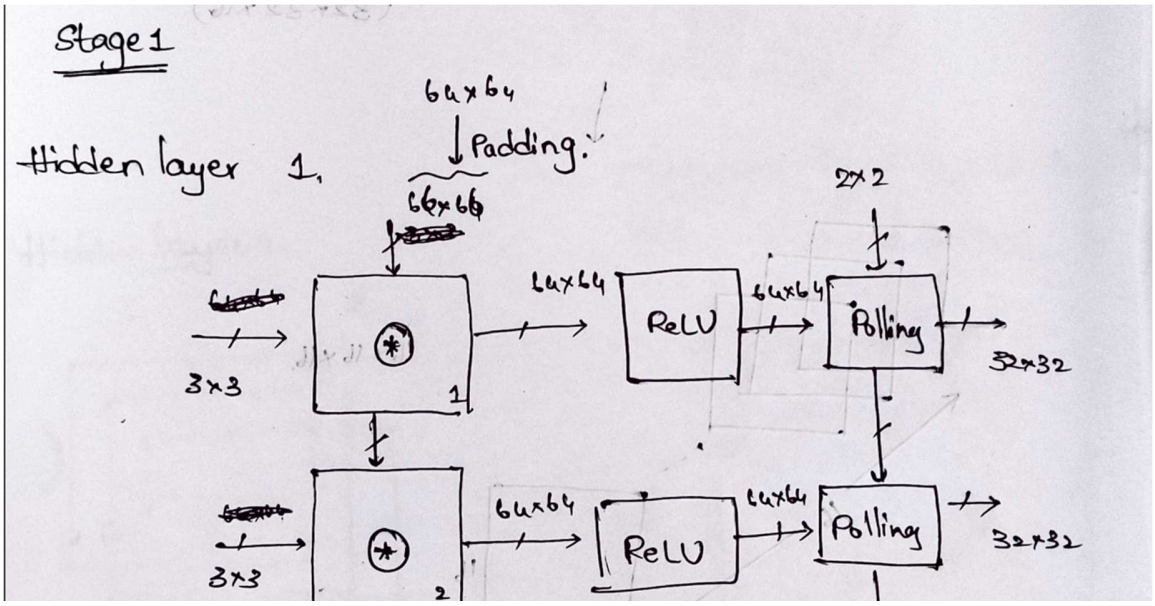


Figure 3.6 Original first-stage parallel-processing design sketch

The original sketch documents the team's reasoning about parallel filters and the flow from convolution through activation to pooling. The final dimensions are 62 × 62 before pooling and 31 × 31 afterward; the diagram's historical annotations are not used as the final tensor specification.

3.11 Valid-signal alignment

The module delays buffer_valid through layer1_valid_d0, layer1_valid_d1 and layer1_valid_d2. These registers accompany arithmetic latency. Pooling counters then select complete spatial windows. A numerically correct MAC evaluated with a valid flag from the wrong cycle can still corrupt a feature map, so data and control alignment is part of functional correctness.

3.12 Second convolution and pooling layer

The layer_2_8 module receives the eight pooled streams from the first layer. Each stream has three 31-element row buffers and a sliding window. These buffers update when layer1_valid is asserted, rather than on every system clock. This distinction is necessary because pooling produces spatially decimated outputs and the stream contains gaps.

The second-layer weights are grouped into two sets of four input channels. Each output filter therefore forms a sum over 36 products. The code registers partial results before combining them with the corresponding bias, reduces the scale by an arithmetic shift and writes the resulting features into second-pooling buffers.

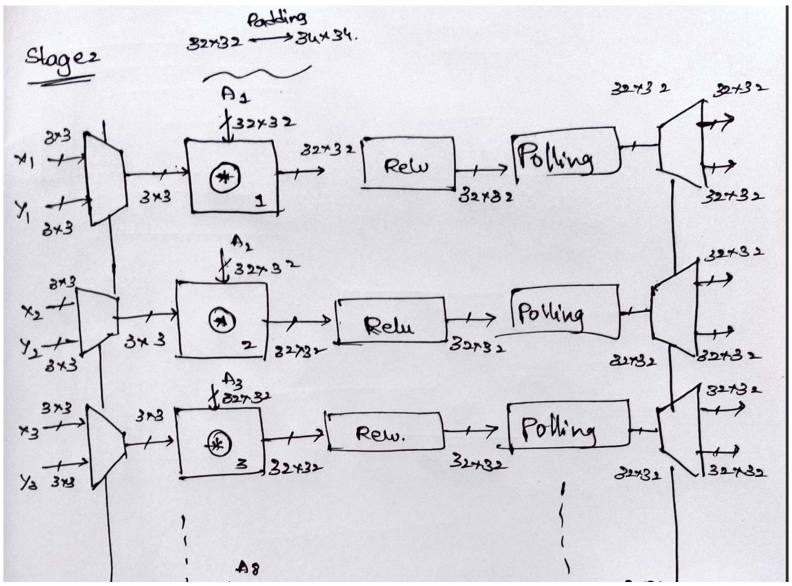


Figure 3.7 Original multichannel second-stage design sketch

The final implementation uses eight output filters and 29-position pooling rows. Its intended second-pooling output is 14 × 14 × 8. The unmatched final row and column of the odd-sized 29 × 29 map do not form complete valid pooling windows. This boundary choice must match the model's MaxPooling2D behavior.

Counters for convolution columns, rows, pooling columns and rows work together with delayed valid signals. The output layer2_valid identifies one spatial position containing eight channel values. Exactly 196 such positions are expected per image, for a total of 1,568 dense inputs.

3.13 Flattening and dense classification

Flattening is implemented as an agreed sequence of features, not as an arithmetic operation. The dense-weight file defines the index as (row × 14 + column) × 8 + channel. Thus the eight channels at one spatial position appear consecutively. The ANN reads eight weights for every valid second-layer output and advances the weight counter by eight.

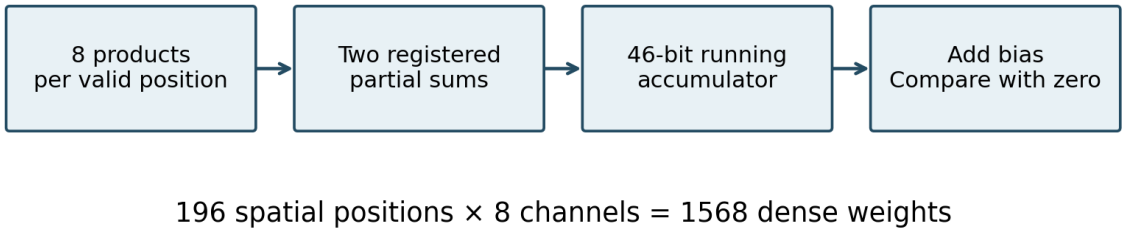


Figure 3.8 Dense accumulation and final binary decision

The ann_8 module calculates eight signed products, forms two registered partial sums and accumulates them into a 46-bit result. The second-layer features have eight fractional bits and the weights have sixteen, so their products have twenty-four. The dense bias is shifted left by eight to align its sixteen fractional bits with the accumulator.

After the final terms have passed through the pipeline, the score is compared with zero. A positive score sets pothole to one; a nonpositive score sets it to zero. For a sigmoid output trained with a 0.5 decision threshold, testing the pre-sigmoid score against zero is mathematically equivalent. The deployed export must retain that same threshold convention.

3.14 Completion and reuse

The ANN uses delayed stop signals to allow in-flight products to reach the accumulator before classification. The rst port is declared but is not used to reset the ANN state in the supplied implementation. Consequently, reliable repeated-frame operation cannot be assumed from this interface alone. A complete restart protocol must reset counters, accumulators, stop flags and valid registers consistently across all modules.

3.15 Module organization and verification

Table 3.5 Active RTL hierarchy

Module or file	Function	Main interface
top_module.sv	Connects the active eight-filter path	clk, reset, ann_res_1, pothole
line_buffer.sv	BRAM reader and input window	window1_1, delay, buffer_valid
layer1_8.sv	Conv 1, activation and Pool 1	Eight 24-bit outputs; layer1_valid
layer_2_8.sv	Grouped Conv 2 and Pool 2	Eight 18-bit outputs; layer2_valid
ann_8.sv	Dense accumulation and threshold	46-bit score; binary pothole
Three active weight headers	Trained integer coefficients	Compile-time arrays

The archive also contains earlier layer_1, layer_2, ann, live, conv and test files. Their presence is useful for tracing development, but their parameters should not be combined indiscriminately with the active top module. In particular, the earlier 16-output second layer and 3,136-weight dense path are different from the final 1,568-feature path.

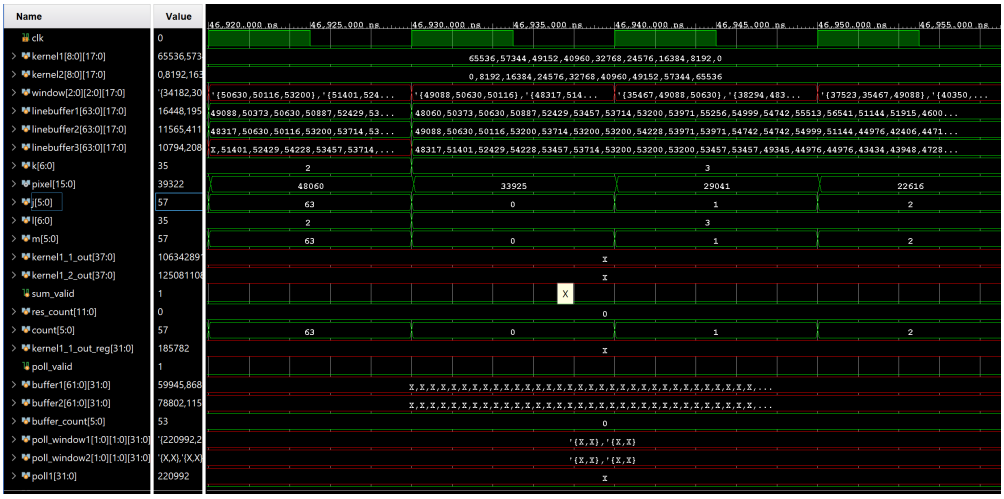


Figure 3.9 Recorded functional simulation of BRAM and row-buffer activity

The earlier simulation records show addresses, pixel values, line buffers and window activity. They document the functional-debugging stage. A screenshot does not establish exhaustive correctness or prove that every final feature matches the trained model; that requires checked reference outputs and valid-count assertions.

3.16 Verification strategy and acceptance checks

Verification is organized around observable stage boundaries. A small manually calculated window is useful for checking coefficient order and signed arithmetic. Full-image tests then check spatial coverage, channel ordering, first and last valid outputs and accumulation completion. A second image without reprogramming is a separate test because initialization and runtime reset are different mechanisms.

Table 3.6 Expected stage counts for the final model

Boundary	Spatial positions	Channels	Values
Input image	4,096	1	4,096
Conv 1	3,844	8	30,752
Pool 1	961	8	7,688
Grouped Conv 2	841	8	6,728
Pool 2	196	8	1,568
Dense score	1	1	1

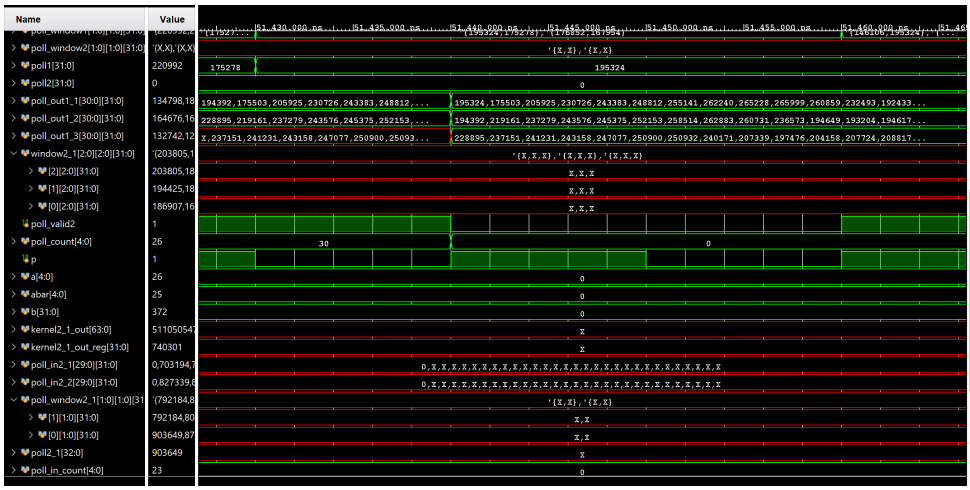


Figure 3.10 Recorded simulation of the stream entering second-layer buffers

Compare initial, boundary and final values from each RTL stage with a fixed-point software model. Include zero and constant images, isolated bright pixels, mixed-sign weights and representative road images. Localize each mismatch before changing several stages.

The reports describe simulation and manual checks. Full-network regression and hardware-versus-software error analysis remain necessary; the waveforms document development rather than complete numerical equivalence.

3.17 Synthesis and resource optimization

Synthesis converts the RTL into logic, registers, arithmetic blocks and memory resources. In this project, the main structural reduction is visible in the move from a larger second layer to eight grouped outputs. Four-channel groups reduce the second-layer connectivity and the classifier receives 1,568 features instead of the earlier 3,136-feature unpadded 16-channel path.

Precision was also reduced between stages. The first pooled outputs use eight fractional bits, and second-layer weights use ten stored bits with eight fractional bits. This reduces operand width relative to an all-Q2.16 approach. The resource report is the evidence for the final mapped cost; a before-and-after synthesis pair would be required to quantify exactly how many resources each change saved.

Name	Slice LUTs (53200)	Slice Registers (106400)	F7 Muxes (26600)	F8 Muxes (13300)	Slice (13300)	LUT as Logic (53200)	LUT as Memory (17400)	Block RAM Tile (140)	DSPs (220)	Bonded IOPADs (130)	BUFGCTRL (32)
cnn_block_design_wrapper	14164	4345	365	2	5139	13160	1004	6.5	220	130	1
cnn_block_design_i (cnn_block_design)	14164	4345	365	2	5139	13160	1004	6.5	220	0	1
axi_smc (cnn_block_design_axi_smc_0	460	510	0	0	197	459	1	0	0	0	0
top_module_0 (cnn_block_design_top	13655	3760	365	2	4925	12653	1002	6.5	220	0	0
inst (cnn_block_design_top_modul	13655	3760	365	2	4925	12653	1002	6.5	220	0	0
u_ann (cnn_block_design_top_n	1828	110	365	2	513	1828	0	0	8	0	0
u_layer_1 (cnn_block_design_to	1188	800	0	0	478	1018	170	4	80	0	0
u_layer_2 (cnn_block_design_to	10580	2797	0	0	3917	9780	800	0	132	0	0
u_line_buffer (cnn_block_desigr	59	53	0	0	32	27	32	2.5	0	0	0

Figure 3.11 Hierarchical utilization from the implemented design

The reported hierarchy allocates 80 DSPs to the first layer, 132 to the second layer and eight to the ANN. Together these consume all 220 device DSP blocks. The second layer also accounts for 10,580 of the 13,655 LUTs within the custom top module, making it the dominant logic consumer. Future parallel expansion therefore faces a DSP-capacity limit even though substantial LUT and BRAM capacity remains.

3.18 Physical implementation

After synthesis, placement assigns cells to physical sites and routing connects them. Pipeline registers shorten logic paths but can increase routing and control complexity. The team's implementation effort therefore includes arithmetic partitioning, data-width management and inspection of timing and placement results, rather than only making the behavioral code compile.

3.19 Timing constraints and clock interpretation

The intended CNN timing target is a 20 ns period, equivalent to 50 MHz. The captured constraint is create_clock -period 20.000 [get_ports clk]. The timing summary reports positive setup and hold slack for the shown analysis. These are useful implementation results, but the timing constraint must describe the actual clock supplied to the circuit [9].

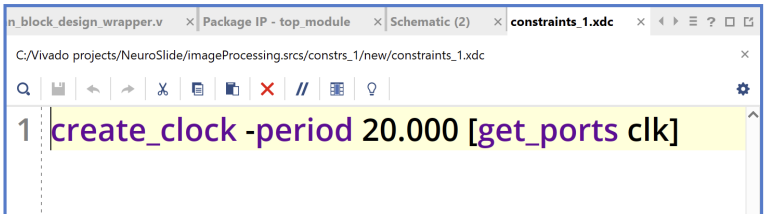


Figure 3.12 Captured 20 ns clock constraint

Clock Summary

Name	Waveform	Period (ns)	Frequency (MHz)
clk_fpga_0	{0.000 5.000}	10.000	100.000
cnn_block_design_i/top_module_0/inst/clk	{0.000 10.000}	20.000	50.000

Figure 3.13 Clock summary containing both 100 MHz and 50 MHz entries

The XSA handoff identifies processing_system7_0_FCLK_CLK0 at 100 MHz and connects that net to both the custom top module and AXI GPIO. No divider is present in the supplied top_module.sv. The screenshot separately lists the CNN clock at 50 MHz. These records must be reconciled before the report can certify operation at a particular frequency.

A timing command does not change the physical frequency. The integration should either configure an actual 50 MHz CNN clock and describe its relationship to the AXI clock, or analyze the CNN correctly at the supplied 100 MHz. Timing checks should then be regenerated from the matching project, and any clock-domain crossing on the result signal should be reviewed. Positive slack under inconsistent constraints is not sufficient proof of the deployed clock rate.

3.20 Custom IP packaging

The accelerator is packaged as xilinx.com:user:top_module:1.0. Packaging preserves the module hierarchy and its dependencies so that the block can be placed into an IP Integrator system [10]. The observed public outputs are ann_res_1[45:0] and pothole, together with clock and reset inputs.



Figure 3.14 Custom IP identity in the packaging interface

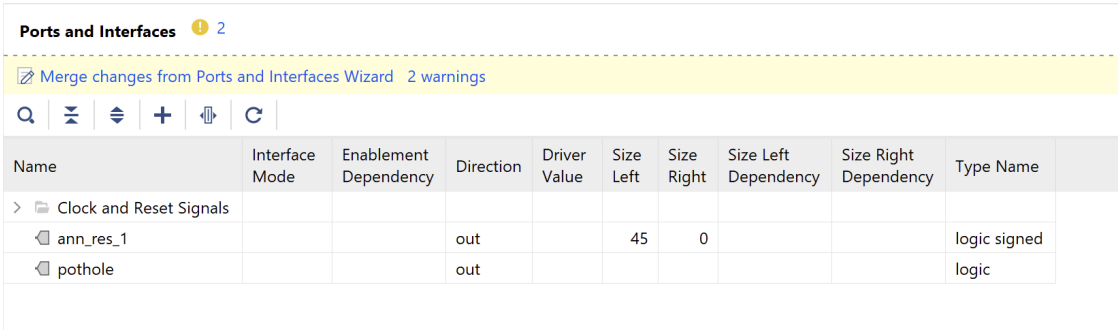


Figure 3.15 Packaged accelerator output ports

The custom CNN block is not itself an AXI slave in the exported design. Instead, its one-bit output is connected to an AXI GPIO peripheral. This is sufficient for a processor-readable demonstration, but it does not create registers for starting a frame, loading pixels, changing weights, reading the full score or checking completion.

The packaging screenshot contains two interface warnings. Their resolution should be recorded before treating the package as a clean reusable release. A reproducible release should include the IP metadata, active RTL, weight headers, Block Memory Generator configuration and image initialization data, rather than relying only on a generated bitstream.

3.21 Processing-system and programmable-logic integration

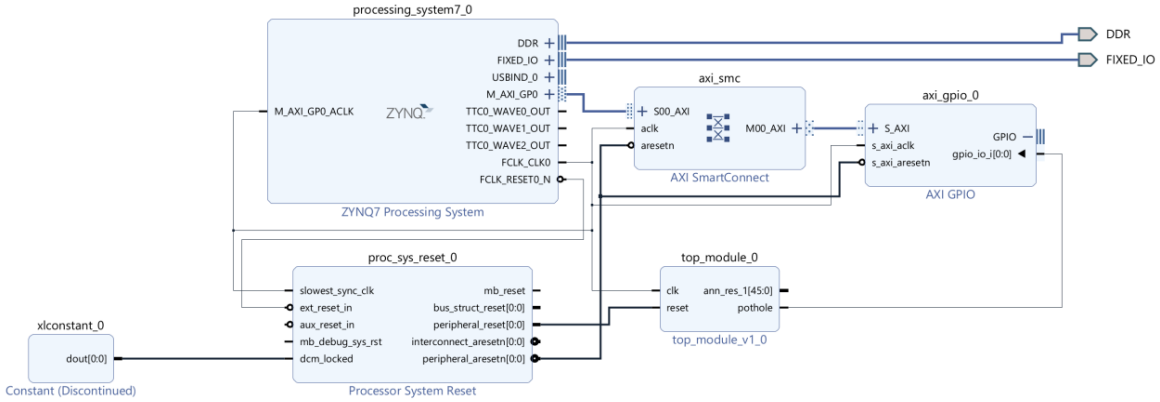


Figure 3.16 Exported system structure with Zynq PS AXI GPIO and CNN IP

The Zynq processing system accesses the AXI GPIO slave through its M_AXI_GP0 port and AXI SmartConnect. The CNN pothole signal drives GPIO channel 1 as a one-bit input. Processor System Reset supplies the peripheral reset signals. The 46-bit ANN score is exposed at the custom IP boundary but is not connected to a software-readable peripheral in the shown block design.

Table 3.7 Hardware handoff values used by software

Item	Value or connection
Device	xc7z020clg484-1 on ZedBoard
AXI GPIO range	0x41200000 to 0x4120FFFF
GPIO channel 1	One bit; all inputs enabled
Classification connection	top_module_0/pothole → axi_gpio_0/gpio_io_i
PS GPIO range	0xE000A000 to 0xE000AFFF
CNN clock in handoff	FCLK_CLK0; metadata reports 100 MHz

The C application uses generated xparameters.h symbols rather than hardcoding these addresses. The SDT branch uses base-address identifiers; the non-SDT branch uses device identifiers. This preserves compatibility with the corresponding generated driver configuration, provided the platform and application are rebuilt from the matching XSA.

3.22 Bitstream export and Vitis workflow

The XSA archive contains cnn_block_design_wrapper.bit, the main hardware handoff, SmartConnect metadata and processing-system initialization files. Its board metadata identifies ZedBoard, and its generation metadata identifies Vivado 2025.2. The launch transcript identifies Vitis/XSDB 2026.1. Recording both versions is important when recreating the platform and application.

A hardware platform is created from the exported XSA, followed by a standalone application for Cortex-A9 core 0. The application includes the AXI GPIO and PS GPIO drivers, initializes the peripherals and reads the hardware classification signal. The programming flow configures the FPGA, initializes the processing system, downloads the application ELF and starts the processor [11].

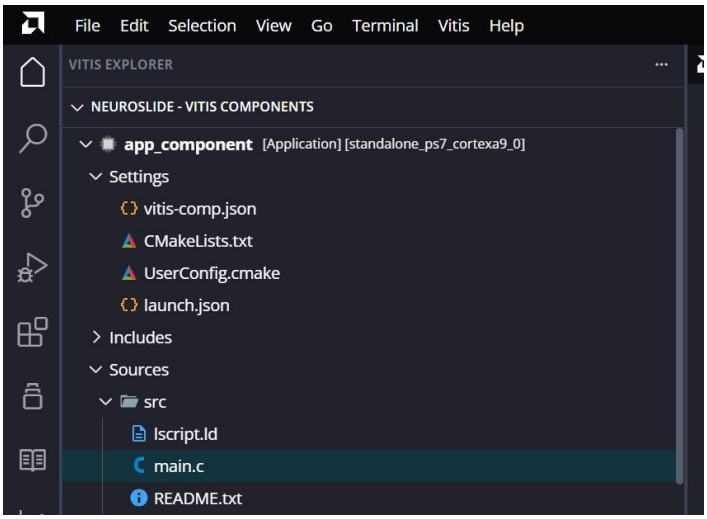


Figure 3.17 Vitis standalone application structure

3.23 Application control flow

The application first initializes the AXI GPIO driver and configures channel 1 as input. It then locates and initializes the PS GPIO driver, sets pin 7 as an output and enables its output driver. The LED is initially written low. After a two-second wait, the application reads the GPIO data and masks it to one bit.

A value of one sets the LED high permanently. A value of zero enters a loop that writes high for 500 ms and low for 500 ms. The classification is not read again inside either loop. Therefore, the visible pattern represents the single sampled result, not continuous classification of a live image stream.

3.24 Embedded software behavior and launch evidence

```
sleep(2);
pothole_result =
    XGpio_DiscreteRead(&CnnGpio, CNN_CHANNEL) & 0x1;

if (pothole_result == 1) {
    XGpioPs_WritePin(&PsGpio, LED_PIN, 1);
    while (1) { sleep(1); }
} else {
    while (1) {
        XGpioPs_WritePin(&PsGpio, LED_PIN, 1);
        usleep(500000);
        XGpioPs_WritePin(&PsGpio, LED_PIN, 0);
        usleep(500000);
    }
}
```

This extract preserves the behavior of the supplied C application. Initialization failures print a diagnostic and return XST_FAILURE. They do not implement a distinct error-pattern state machine, and the error branches occur before the normal LED initialization is complete. An LED that stops blinking is therefore not a reliable encoded fault indicator in this version.

Table 3.8 Deployment events recorded by XSDB

Event	**Observed outcome**
FPGA programming	Bitstream transfer reached 100%
PS initialization	ps7_init and ps7_post_config issued without a reported error
ELF download	Application downloaded successfully; start address 0x00100000
Processor launch	ARM Cortex-A9 core 0 reported Running

The launch record proves that programming and application launch completed. It does not contain a timed inference measurement or the application’s printed CNN result. Hardware photographs provide complementary evidence of the physical setup, but a still image cannot prove the temporal blinking sequence or the correctness of a classifier output.

CHAPTER 4

CONTRIBUTION OF EACH STUDENT

The continuing project team comprises Venkata Durgesh, Gurudeep Sai and Manudeep. The completed work spans model development, RTL design, verification, physical implementation, system integration and hardware demonstration. The individual allocation below is reserved for the team to confirm before submission; ownership of specific technical tasks should not be assigned without that confirmation.

4.1 Venkata Durgesh 230907454

Venkata Durgesh was primarily responsible for the software-based development of the CNN model and its integration into the FPGA-based system. His contributions included image preprocessing using OpenCV and Python, dataset collection and preparation, and designing the CNN architecture through multiple experimental iterations. He performed model training, testing, and performance evaluation, followed by trained weight and bias extraction and their conversion into fixed-point representations suitable for FPGA implementation. He also generated the required memory initialization files (.coe) for BRAM-based image storage and supported the verification of hardware results against Python-based model predictions.

In the hardware integration phase, he was responsible for custom IP packaging, designing the Vivado Block Design, integrating the CNN accelerator with the Zynq Processing System through PS–PL interfaces, and generating the FPGA bitstream. He also worked on exporting the hardware platform to Vitis, FPGA hardware implementation, and programming the ZedBoard for system-level validation.

4.2 Gurudeep Sai 230907202

Gurudeep Sai was primarily responsible for translating the trained CNN architecture into an efficient hardware design using SystemVerilog. He developed the complete hierarchical RTL architecture, including convolutional layers, line buffers, activation functions, max-pooling operations, and the final ANN classification stage. He implemented fixed-point arithmetic throughout the datapath and performed extensive simulation-level debugging and layer-by-layer functional verification. Intermediate RTL outputs were compared with Python-generated reference results to establish computational correctness and identify numerical discrepancies.

He further optimized the fixed-point architecture, arithmetic bit widths, and hardware datapath to improve FPGA resource utilization. His work included RTL synthesis, analysis of LUT, DSP, BRAM, and register utilization, and resolving synthesis and implementation issues. He also worked on timing optimization, placement and routing challenges, and achieving an

implementable hardware architecture within the available ZedBoard resources. Additionally, he contributed to custom IP packaging, PS–PL integration, and successful bitstream generation.

4.3 Manudeep 230907436

Manudeep was primarily responsible for embedded software development and software–hardware interfacing on the ZedBoard. His contributions included creating the hardware platform in Vitis using the exported Xilinx Support Archive (.xsa), configuring the standalone ARM processing environment, and developing the embedded C application for accessing the CNN classification output through the AXI GPIO interface.

He worked on LED interfacing to provide a visual indication of the detected road condition, including continuous illumination for pothole detection and blinking operation for normal road classification. He also contributed to output testing, debugging the embedded application, and comparing the classification results obtained from the FPGA RTL implementation with the corresponding Python model outputs to support end-to-end system verification.

4.4 Joint integration work

The final integration and hardware validation of the CNN-based pothole detection system were carried out collaboratively by all three team members. The integration involved combining the Python-trained CNN model, SystemVerilog RTL architecture, and embedded ARM software into a complete FPGA-based system. The team jointly addressed compatibility between software-generated parameters and hardware fixed-point representations, verified the consistency of intermediate layer outputs, and resolved issues arising during the transition from functional simulation to FPGA implementation.

The team collectively worked on integrating the CNN accelerator as a custom IP within the Vivado Block Design, establishing communication between the Programmable Logic (PL) and Processing System (PS) through the AXI GPIO interface, and configuring the clock and reset architecture. Collaborative efforts were also directed toward resolving IP packaging issues, synthesis and implementation errors, resource constraints, timing requirements, and bitstream generation.

During the final hardware validation phase, the team worked together on exporting the hardware platform to Vitis, deploying the CNN accelerator on the ZedBoard, developing the software–hardware interface, and evaluating the classification outputs against Python reference results. The integration of BRAM-based image input, hardware CNN inference, ARM-based result acquisition, and LED-based output indication represented the combined efforts of the team in developing a complete hardware–software co-design solution for pothole detection.

CHAPTER 5

RESULT ANALYSIS

5.1 Software classification result

The final notebook capture reports 89.20% test accuracy over 750 images, evenly divided between normal road and pothole. The confusion matrix contains 322 correct normal predictions and 347 correct pothole predictions. The total correct count is 669, giving 669/750 = 0.8920.

```
Test Accuracy: 89.20%

Classification Report:
              precision    recall  f1-score   support

      Normal     0.9200    0.8587    0.8883       375
     Pothole     0.8675    0.9253    0.8955       375

    accuracy                         0.8920       750
   macro avg     0.8938    0.8920    0.8919       750
weighted avg     0.8938    0.8920    0.8919       750
```

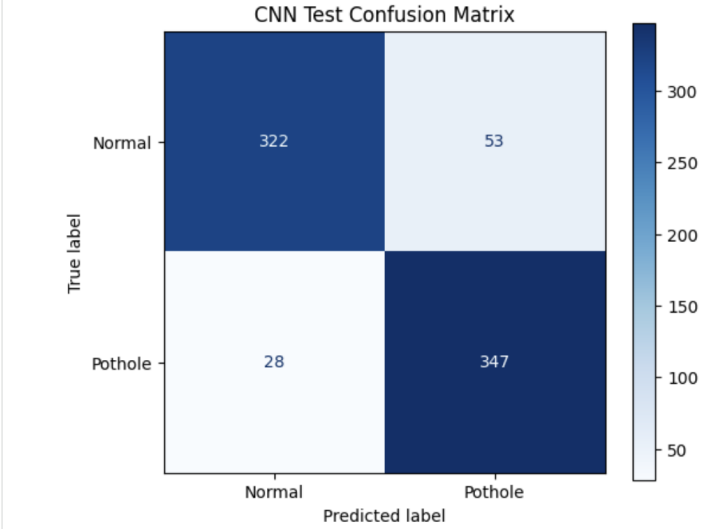


Figure 5.1 Software test metrics and confusion matrix

There are 53 normal-road images predicted as potholes and 28 pothole images predicted as normal. The higher pothole recall of 92.53% is useful for finding positive cases, but false negatives remain. This result describes image classification on the recorded test split and should not be interpreted as deployment accuracy across all roads, weather conditions or camera positions.

5.2 Interpretation of model quality

Table 5.1 Class-wise software performance

Class	Precision	Recall	F1 score	Support
Normal	0.9200	0.8587	0.8883	375
Pothole	0.8675	0.9253	0.8955	375
Macro average	0.8938	0.8920	0.8919	750
Weighted average	0.8938	0.8920	0.8919	750

Taking pothole as the positive class, precision is 347/(347 + 53) = 86.75%, while recall is 347/(347 + 28) = 92.53%. The false-positive rate is 53/375 = 14.13%, and the false-negative rate is 28/375 = 7.47%. The F1 score balances precision and recall; it does not describe the severity of each missed pothole.

The balanced test support prevents class frequency alone from inflating accuracy toward a dominant class. It does not establish that training and test images are independent. A dataset release should identify source images, split rules and any near-duplicate removal, especially if consecutive frames from the same scene are present.

5.3 Relationship to fixed-point hardware

The software result validates the compact model as a learning configuration. The RTL uses quantized weights, truncated intermediate values and a finite-width accumulator. Its classification accuracy must be measured separately by feeding the same test images into a validated fixed-point reference and the FPGA implementation.

A useful comparison records floating-point score, fixed-point score, hardware score and class for each image. Agreement on a few examples is a bring-up result; agreement across the complete split supports a numerical validation claim. Images close to the zero-score decision boundary deserve particular attention because small quantization errors can change the output class.

5.4 Model size

The software summary reports 1,945 parameters and 7.60 KB at its displayed storage precision. The integer coefficient files use mixed widths, so their logical coefficient storage differs from that display. Parameter count is also distinct from FPGA resource use: the design may replicate arithmetic or buffers to increase parallelism even when the trained model is small.

5.5 FPGA utilization and bottlenecks

Table 5.2 Implemented device utilization

Resource	Used	Available	Utilization
Slice LUTs	14,164	53,200	26.62%
Slice registers	4,345	106,400	4.08%
BRAM tiles	6.5	140	4.64%
DSP blocks	220	220	100.00%
Slices	5,139	13,300	38.64%

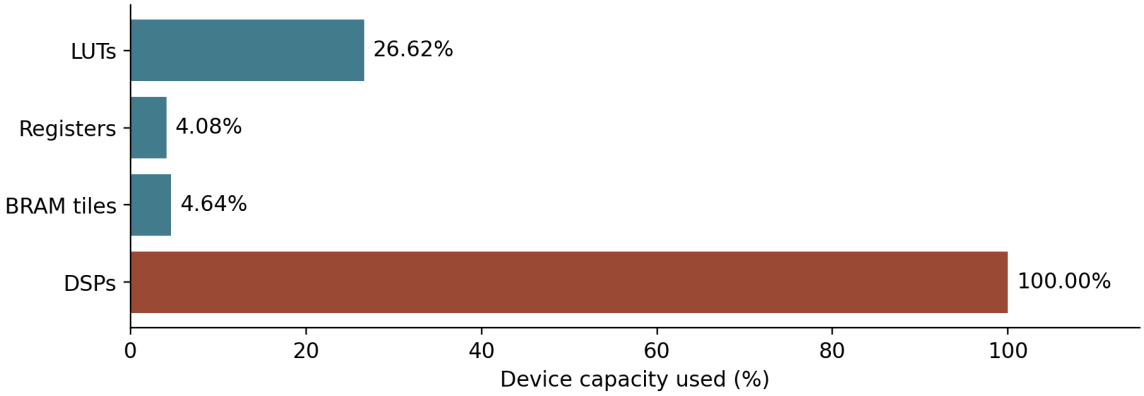


Figure 5.2 Resource utilization calculated from the implementation screenshot

The DSP budget is fully occupied. This limits further replication of multipliers even though the LUT and register percentages are relatively modest. The result supports the need for grouped connectivity and a careful precision choice, but it does not show the isolated resource saving of each optimization because paired baseline implementation reports are not available.

Table 5.3 Arithmetic distribution in the custom hierarchy

Block	LUTs	Registers	DSPs
Layer 1	1,188	800	80
Layer 2	10,580	2,797	132
ANN	1,828	110	8
Input line buffer	59	53	0

The second layer dominates logic and uses 60% of the device DSPs. Time multiplexing, additional pipelining or different arithmetic mappings are possible future tradeoffs, but each must be evaluated against throughput and numerical accuracy.

5.6 Timing results and their scope

Design Timing Summary

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	1.626 ns	Worst Hold Slack (WHS):	0.033 ns	Worst Pulse Width Slack (WPWS):	4.020 ns
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	0.000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	24733	Total Number of Endpoints:	24733	Total Number of Endpoints:	5561

All user specified timing constraints are met.

Figure 5.3 Reported implementation timing summary

Clock	Edges (WNS)	WNS (ns)	TNS (ns)	Failing Endpoints (TNS)	Total Endpoints (TNS)	Edges (WHS)	WHS (ns)	THS (ns)	Failing Endpoints (THS)	Total Endpoints (THS)	WPWS (ns)	TPWS (ns)
clk_fpga_0	rise - rise	2.884	0.000	0	1255	rise - rise	0.121	0.000	0	1255	4.020	0.000
cnn_block_design_i/top_module_0/ir	rise - rise	1.626	0.000	0	23463	rise - rise	0.033	0.000	0	23463	8.750	0.000

Figure 5.4 Per-clock setup and hold results

Table 5.4 Reported timing values

Metric	Value	Meaning
Worst setup slack	1.626 ns	Positive under the shown constraints
Total negative slack	0.000 ns	No reported setup violation
Worst hold slack	0.033 ns	Positive but small hold margin
Setup and hold failing endpoints	0	For the reported analysis
Worst pulse-width slack	4.020 ns	Positive in the aggregate summary

The timing summary states that all user-specified constraints are met. This is a meaningful physical-design milestone for that constraint set. It is not equivalent to measuring the operating clock or establishing frame throughput. The 50 MHz CNN constraint and 100 MHz handoff connection described in Section 3.19 must be made consistent before a final timing sign-off.

The small positive hold slack should be preserved when modifying the design. Any change to clocking, arithmetic width, pipeline registers or placement can change the result, so the final implementation reports must correspond to the exact bitstream used for the demonstration.

5.7 Estimated power and implementation warnings

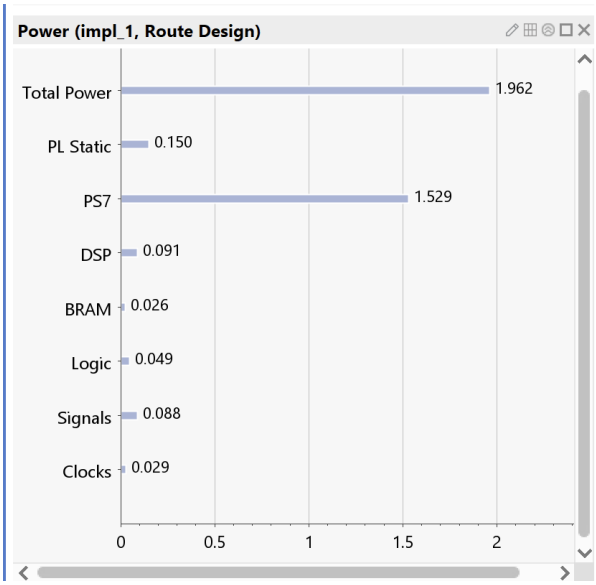


Figure 5.5 Vivado implementation power estimate

The displayed total power estimate is 1.962 W. The largest displayed contribution is PS7 at 1.529 W. The other displayed components include 0.150 W of PL static power, 0.091 W for DSP, 0.026 W for BRAM, 0.049 W for logic, 0.088 W for signals and 0.029 W for clocks. Their rounded values sum to the shown total.

This is a tool estimate, not a measured board-input power result. Its accuracy depends on activity assumptions, clock settings and device conditions. It should not be converted into a measured energy-per-frame claim while throughput and the final clock configuration remain unverified.

Table 5.5 Warning counts visible in the implementation record

Report	Visible count	Interpretation
DRC	112 warnings	Requires review of the underlying rule messages
Methodology	4 critical warnings and 2 warnings	Timing success does not resolve these automatically
IP ports and interfaces	2 warnings	Packaging issues require inspection
SSN	Status No Analysis	Not a completed signal-noise validation

These counts do not identify the causes of the warnings. They are retained as part of the implementation status rather than described as a warning-free design. Final release review should record which messages were corrected, which are accepted and why.

5.8 Physical hardware demonstration

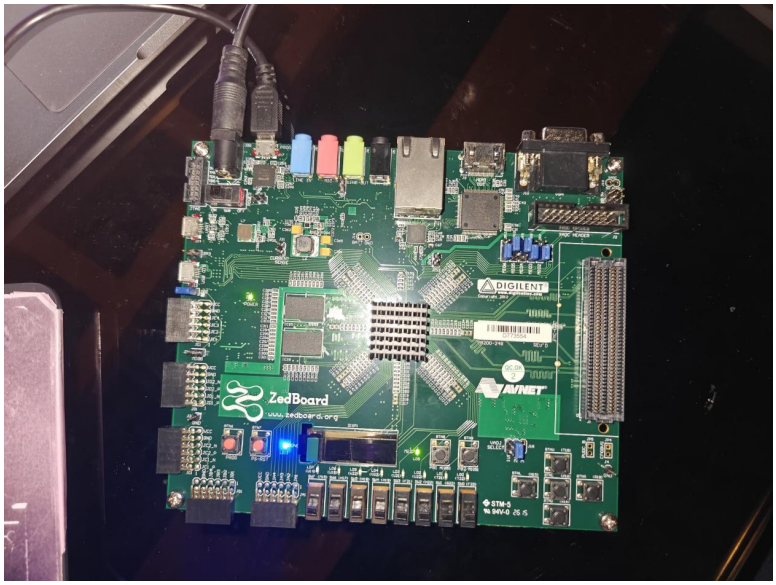


Figure 5.6 ZedBoard connected for the hardware demonstration

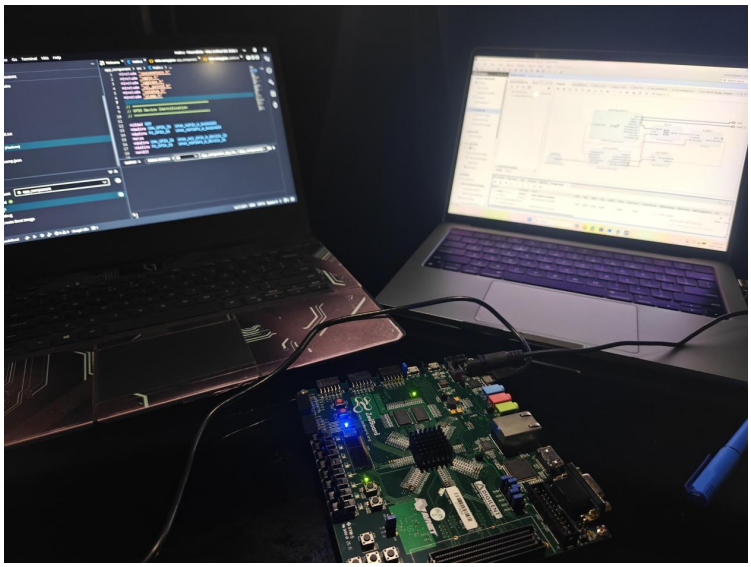


Figure 5.7 Development computers and FPGA during deployment

The photographs document the board and development setup. Together with the XSA and XSDB log, they support the reported deployment milestone. The C application defines a steady LED for a sampled pothole result and blinking for a sampled normal-road result. A video or timestamped GPIO trace would be required to independently verify the blink period and repeated behavior.

5.9 Throughput target and analytical timing budget

At 240 FPS, the permitted interval between completed frames is 1/240 s, or approximately 4.167 ms. At a genuine 50 MHz CNN clock, this corresponds to 50,000,000/240 = 208,333.3 clock cycles per frame on average. This is a target budget and not an observed inference count.

Table 5.6 Analytical performance quantities

Quantity	Value	Interpretation
Frame interval at 240 FPS	4.167 ms	Required average completion interval
50 MHz cycle budget	208,333 cycles per frame	Approximate available cycles
One-pixel-per-cycle input scan	4,096 cycles; 81.92 µs	Ideal input scan only; excludes fill and drain
Input rate at 240 FPS	983,040 pixels/s	64 × 64 pixels per frame
16-bit image data rate	1.966 MB/s	Payload only; decimal MB
Model multiplication demand	124.93 million/s	520,544 multiplications × 240 frames/s

The ideal scan time is much shorter than the frame target, but it is not a full-network latency measurement. It excludes memory startup, convolution and pooling boundary gaps, ANN completion, frame restart and host-side processing. A clock frequency by itself therefore cannot demonstrate the FPS requirement.

5.10 Measurement method

The next performance test should expose start, done and a cycle counter. Measure first-input acceptance to valid final result for latency, then measure the interval between completed frames during a long repeated run for throughput. If preprocessing or transfer is included in the intended application, report a second end-to-end measurement that includes those costs.

The two-second sleep in the current software is a conservative demonstration delay, not inference latency. The single GPIO read and infinite LED loop prevent the application from establishing sustained throughput. A continuous test harness must replace that behavior before reporting an achieved frame rate.

5.11 Deviations and verification priorities

The final result is a compact grouped binary model, a modular hardware implementation and a deployed PS–PL demonstration. Several checks remain necessary before treating the demonstrator as a validated continuous image-processing system. They are specific to the observed code and integration rather than generic assumptions about FPGA design.

Table 5.7 Priority checks before final technical sign-off

Item	Observed reason	Required check
Clock consistency	Handoff 100 MHz versus CNN constraint 50 MHz	Align physical clock and timing model
Runtime restart	ANN rst is unused; several pipeline states rely on initialization	Verify two consecutive independent frames
Bias scaling	Layer 1 expression includes raw and shifted bias terms	Compare against bit-accurate intended equation
Intermediate precision	Conv 2 uses 22-bit partial registers after wider-operand arithmetic	Check truncation and overflow on test data
Final valid counts	Complex row, column and delayed-valid control	Assert 961 and 196 pooled spatial outputs
Result readiness	Software waits a fixed two seconds; no done register	Add completion flag and timeout
Numerical equivalence	Software accuracy is separate from RTL behavior	Run identical images and compare scores/classes

The raw and shifted first-layer bias terms deserve inspection: adding a bias already expressed at one scale to another aligned representation introduces a contribution beyond the usual single bias term. The magnitude may be small after shifting, but its effect should be checked rather than assumed irrelevant. Similarly, narrowing partial sums can be acceptable only when the range and quantization behavior are deliberate.

The source archive lacks the complete original build configuration and input-memory initialization. A bitstream can demonstrate a programmed design, but rebuilding and independently verifying the design requires those source dependencies. Final sign-off should pair the exact source revision with the tested bitstream and recorded results.

CHAPTER 6

CONCLUSION AND FUTURE SCOPE

6.1 Summary of the work

NeuroSlide develops a complete engineering path from a small image-classification model to a Zynq FPGA demonstrator. The final architecture accepts 64 × 64 grayscale inputs, uses eight first-layer filters and grouped second-layer convolution, reduces the representation to 1,568 features and forms one binary classification score. Its software evaluation reports 89.20% accuracy on a balanced 750-image test set.

The hardware work expresses this model through BRAM access, line buffers, sliding windows, fixed-point arithmetic, registered partial sums, pooling and dense accumulation. The modular structure exposes data and valid signals between stages. Custom IP packaging and AXI GPIO integration connect the result to the processing system, where a standalone application drives a visible LED indication.

6.2 Major contributions

The main contributions are the hardware-oriented model reduction, the explicit streaming data path, the integration of trained integer coefficients, and the extension from simulation into physical implementation and software deployment. The implementation record reports positive timing slack under the shown constraints and identifies the resource distribution of the accelerator.

The design uses all 220 DSP blocks on the selected device. This makes the grouped architecture and arithmetic choices practically relevant rather than purely theoretical. The second layer is the dominant logic consumer, providing a clear direction for future resource optimization.

6.3 Conclusions and limitations

The prototype establishes a useful foundation for FPGA-based pothole classification. It does not yet establish measured 240 FPS throughput, full-test-set FPGA accuracy, warning-free implementation or reliable continuous-frame operation. The clock-source and timing-constraint discrepancy must be resolved before confirming the final operating frequency. These distinctions keep the demonstrated integration achievements separate from performance and numerical-validation goals.

The current application demonstrates result access rather than a complete road-monitoring product. A live sensor path, per-frame control, robust error reporting and broader environmental evaluation are required before the approach can be considered for a real deployment.

6.4 Future scope

6.4.1 Complete the numerical and clock validation

First, archive the exact trained model and reproduce its inference with the same fixed-point scales, truncation rules and feature order as the RTL. Compare all stage outputs, resolve bias and overflow questions, then evaluate final class agreement on the test split. In parallel, reconcile the physical CNN clock with the constraints and regenerate timing, clock-interaction and implementation reports.

6.4.2 Add a reliable frame protocol

Introduce start, busy, done and error status registers or equivalent signals. Reset every frame-dependent register deterministically and verify consecutive images without reprogramming the FPGA. Replace the fixed software wait with completion polling or an interrupt and a bounded timeout. Separate normal-road, pothole and error indications explicitly.

6.4.3 Measure sustained throughput and energy

Use hardware cycle counters for accelerator latency and frame interval, and a host timer for end-to-end throughput. Run enough frames to exclude one-time setup costs from steady-state results while still reporting those costs separately. Measure board-input power under a defined workload before calculating energy per frame.

6.4.4 Extend the input and resource architecture

A processor-writable image buffer, DMA path or camera interface would remove the fixed-BRAM-image limitation. Because all DSP blocks are used, future optimization can explore controlled time multiplexing, reduced-width arithmetic with retraining, or additional pipeline balancing. Every alternative should be evaluated on accuracy, timing and measured frame rate together.

6.4.5 Improve application coverage

Broader datasets should include road surfaces, shadows, water, lighting changes and camera viewpoints that challenge the classifier. If the intended task requires pothole location or size, the output model and annotation scheme must change accordingly. Multi-board portability should be assessed through separately documented builds and tests rather than inferred from successful IP packaging.

VISIBLE OUTCOME

The project has produced a trained compact CNN configuration, SystemVerilog modules and integer parameter files, functional-simulation records, an implemented Zynq design, a hardware export containing a bitstream, and a standalone application that reads the classifier output and controls an LED. The hardware photographs and launch transcript document the physical demonstration stage.

Table 6.1 Visible technical deliverables

Deliverable	Recorded outcome
Model	Grouped eight-filter CNN; 1,945 parameters; 89.20% software test accuracy
RTL	Input window, two convolution/pooling stages and dense classifier
Implementation	Utilization, timing, power-estimate and warning screenshots
Hardware export	XSA with ZedBoard identity, handoff and bitstream
Embedded application	AXI GPIO read with steady or blinking LED behavior
Physical demonstration	Board photographs and successful programming/launch transcript

Publication status: [Enter any actual conference-paper submission, acceptance or publication details, if applicable. No publication claim is made here.]

A future paper or demonstration submission should include measured frame rate, the final clock configuration, fixed-point versus floating-point accuracy, complete resource and power conditions, and a reproducible software and hardware package. These measurements would make the project's performance claims independently assessable.

REFERENCES

[1] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. MIT Press, 2016. Online: deeplearningbook.org.

[2] H. Maeda, Y. Sekimoto, T. Seto, T. Kashiyama, and H. Omata, “Road Damage Detection and Classification Using Deep Neural Networks with Smartphone Images,” Computer-Aided Civil and Infrastructure Engineering, vol. 33, no. 12, pp. 1127–1141, 2018, doi: 10.1111/mice.12387.

[3] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, “Efficient Processing of Deep Neural Networks: A Tutorial and Survey,” Proceedings of the IEEE, vol. 105, no. 12, pp. 2295–2329, 2017, doi: 10.1109/JPROC.2017.2761740.

[4] Y.-H. Chen, J. Emer, and V. Sze, “Eyeriss: A Spatial Architecture for Energy-Efficient Dataflow for Convolutional Neural Networks,” in Proc. 43rd Annual International Symposium on Computer Architecture, 2016, pp. 367–379, doi: 10.1145/3007787.3001177.

[5] Y. Umuroglu et al., “FINN: A Framework for Fast, Scalable Binarized Neural Network Inference,” in Proc. ACM/SIGDA International Symposium on Field-Programmable Gate Arrays, 2017, pp. 65–74. Preprint: arXiv:1612.07119.

[6] B. Jacob et al., “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference,” in Proc. IEEE Conference on Computer Vision and Pattern Recognition, 2018, pp. 2704–2713. Preprint: arXiv:1712.05877.[7] AMD, Zynq 7000 SoC Technical Reference Manual, UG585. AMD Technical Information Portal, docs.amd.com, accessed Oct. 27, 2026.

[8] AMD, AXI GPIO LogiCORE IP Product Guide, PG144, ver. 2.0. AMD Technical Information Portal, docs.amd.com, accessed Oct. 27, 2026.

[9] AMD, Vivado Design Suite User Guide: Using Constraints, UG903. AMD Technical Information Portal, docs.amd.com, accessed Oct. 27, 2026.

[10] AMD, Vivado Design Suite User Guide: Creating and Packaging Custom IP, UG1118. AMD Technical Information Portal, docs.amd.com, accessed Oct. 27, 2026.

[11] AMD, Vitis Unified Software Platform Documentation: Embedded Software Development, UG1400. AMD Technical Information Portal, docs.amd.com, accessed Oct. 27, 2026.

MINOR SPECIALIZATION PROJECT DETAILS

Student name	**Venkata Durgesh**
Registration number	230907454
Branch / Section / Roll number	ECE – D, Roll Number 46
Mail ID / Mobile	venkatadurgesh03@gmail.com / 8074783630

Student name	**Gurudeep Sai**
Registration number	230907202
Branch / Section / Roll number	ECE – D, Roll Number 21
Mail ID / Mobile	vgurudeepsai@gmail.com / 8977746699

Student name	**Manudeep**
Registration number	230907436
Branch / Section / Roll number	ECE – C, Roll Number 39
Mail ID / Mobile	manudeeptirunagari@gmail.com / 9949891613

Project and guide	**Details**
Project title	NeuroSlide FPGA CNN Accelerator for Pothole Detection
Minor specialization	Minor in Embedded Systems
Guide name and designation	Dr. Rahul Ratnakumar, Assistant Professor
Guide department	School of Electrical Engineering
Guide mail ID / Mobile	rahul.ratnakumar@manipal.edu / 7891400609

Signature of students
M Venkata Durgesh: _____
V Gurudeep Sai: _____
T Manudeep: _____

Signature of guide: _____

Date: _____

NeuroSlide_Final_Project_Report.pdf

ORIGINALITY REPORT

2%	2%	1%	1%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	Submitted to Manipal Academy of Higher Education (MAHE) Student Paper	<1%
2	www.coursehero.com Internet Source	<1%
3	Submitted to RMIT University Student Paper	<1%
4	addi.ehu.es Internet Source	<1%
5	www.diva-portal.org Internet Source	<1%
6	Lafrance, Samantha. "A Multi-Scale Fusion CNN-ViT Hybrid Architecture for Semiconductor Wafer Defect Detection.", Florida Agricultural and Mechanical University Publication	<1%
7	"IPTA 2018 Proceedings", 2018 Eighth International Conference on Image Processing Theory, Tools and Applications (IPTA), 2018 Publication	<1%
8	5dok.net Internet Source	<1%
9	dspace.ewubd.edu Internet Source	<1%
10	sciencedocbox.com Internet Source	<1%
11	ris.unimap.edu.my Internet Source	<1%
12	bpb-us-w2.wpmucdn.com Internet Source	<1%
13	china.xilinx.com Internet Source	<1%
14	eprints.umm.ac.id Internet Source	

		<1%
15	gopher.sbaonline.sba.gov Internet Source	<1%
16	reference-global.com Internet Source	<1%
17	scholarworks.utrgv.edu Internet Source	<1%

Exclude quotes	On	Exclude matches	< 3 words
Exclude bibliography	On		